

```
1. # Create a constant array
    # Prints "[[ 7.  7.]
    #           [ 7.  7.]]"
```

```
2. # Create an array filled with random values
    # Might print "[[ 0.91940167  0.08143941]
    #               [ 0.68744134  0.87236687]]"
```

3. Create a NumPy array containing the numbers from 10 to 50. Display:

The array
Number of elements
Shape
Dimension
Data type

4. Using NumPy, create:

A 1D array containing 10 zeros
A 3 × 3 array containing ones
A 4 × 4 identity matrix
An array containing numbers from 5 to 50 with a step size of 5.

5. Create an array containing numbers from 1 to 24. Reshape it into:

4 × 6
6 × 4
2 × 3 × 4

Display the shape and ndim of each result.

6. Consider: `marks = np.array([78, 85, 92, 67, 88, 73, 95, 81])`

Write NumPy statements to display:

First element
Last element
Third element
First four elements
Last three elements

7. Create:

```
a = np.array([10, 20, 30, 40, 50])
b = np.array([5, 10, 15, 20, 25])
```

Calculate:

a + b
a - b
a * b
a / b

Square of every element in a

8. Given:

```
salary = np.array([25000, 32000, 28000, 45000,  
                   50000, 38000, 42000, 30000])
```

Find:

Total salary
Average salary
Highest salary
Lowest salary
Median salary
Standard deviation

9. Given: marks = np.array([45, 78, 89, 32, 67, 90, 55, 76, 41, 84])

Using NumPy boolean indexing, find:

Marks greater than 75
Marks between 50 and 80
Number of students who scored below 50
Percentage of students who scored 60 or above

Do not use a Python for loop.

10. The following array contains marks of four students in three subjects:

```
marks = np.array([  
    [80, 75, 90],  
    [65, 70, 72],  
    [88, 92, 85],  
    [55, 60, 58]  
])
```

Calculate:

Total marks of each student
Average marks of each student
Average marks in each subject
Student with the highest total
Highest mark in each subject.

11. Create:

```
A = np.array([  
    [1, 2],  
    [3, 4]  
])
```

```
B = np.array([  
    [5, 6],  
    [7, 8]  
])
```

```
)
```

Perform:

Matrix addition
Element-wise multiplication
Matrix multiplication
Transpose of A.

explain the difference between:
A * B and A @ B

12. Daily temperatures for two weeks are:

```
temperature = np.array([
    31, 32, 33, 35, 36, 34, 32,
    30, 29, 31, 33, 35, 37, 36
])
```

Find:

Mean temperature
Maximum and minimum temperatures
Number of days above the average temperature
Temperatures greater than 34°C
Difference between maximum and minimum
Standard deviation

13. Monthly sales of a company are:

```
sales = np.array([
    12000, 15000, 17000, 14000,
    18000, 21000, 19000, 22000,
    25000, 23000, 26000, 28000
])
```

Determine:

Total annual sales
Average monthly sales
Month with the highest sales
Month with the lowest sales
Number of months with sales above ₹20,000
Percentage increase from January to December

Hint: Remember that NumPy indexing starts at 0, while month numbering normally starts at 1.

14. Consider marks of five students in four subjects:

```
marks = np.array([
    [78, 85, 90, 88],
    [65, 70, 72, 68],
    [92, 88, 95, 91],
    [55, 60, 58, 62],
])
```

```
[80, 75, 82, 79]
])
```

Write a NumPy program that calculates:

Total marks of every student
Average of every student
Average mark for every subject
Highest-scoring student
Lowest-scoring student
Students whose average is greater than 80
Overall class average

Then assign a performance category based on average:

Average	Category
≥ 85	Excellent
70-84.99	Good
50-69.99	Average
< 50	Needs Improvement

Challenge: Try assigning these categories using `np.where()` or `np.select()` instead of a Python loop.

15. Generate 20 random integers between 1 and 100.
16. Find all even numbers from the generated array.
17. Replace values below 50 with 0.
18. Sort the array in ascending and descending order.
19. Find unique values and their frequencies.
20. Create a 5 × 5 random matrix and calculate row-wise and column-wise means.
21. Take your own example to normalize an array.
22. use your own data to standardize an array.
23. Create a NumPy array containing 100 random exam marks and determine how many students fall into the ranges 0-39, 40-59, 60-79, and 80-100.
24. Compare the time required to multiply 1,000,000 numbers using a Python list and a NumPy array.